

Modulo 2

Calculito

2026-04-08

Introducción

Este es un documento en R Markdown. Permite generar HTML o Word combinando texto y código. El lenguaje R es una poderosa herramienta matemática y estadística. En este reporte generado con R Markdown, analizaremos la resolución de distintos problemas y evaluaremos la calidad de diferentes algoritmos implementados.

Probando primeros comandos de R

R es un lenguaje muy parecido a matlab y trabaja con variables que en general pueden ser matrices.

```
A <- 38
A <- 40
B <- 60
C <- B - A
C
```

```
## [1] 20
```

Uso de data sets o base de datos internos de R

Usando el comando `data()` en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan cargados en tablas de las cuales podemos elegir columnas especificas si escribimos el signo `$` despues del comando de informacion especifico que queremos.

También está el comando `summary()`, que es una de las herramientas más útiles y rápidas que tiene R para hacer estadística descriptiva básica.

```
# Planteamos el dataset
data(cars)
```

```
# Usamos el símbolo $ para extraer SOLO la columna de velocidades (speed)
velocidades <- cars$speed
```

```
# Mostramos los primeros 6 valores de esa columna aislada
head(velocidades)
```

```
## [1] 4 4 7 7 8 9
```

```
summary(cars)
```

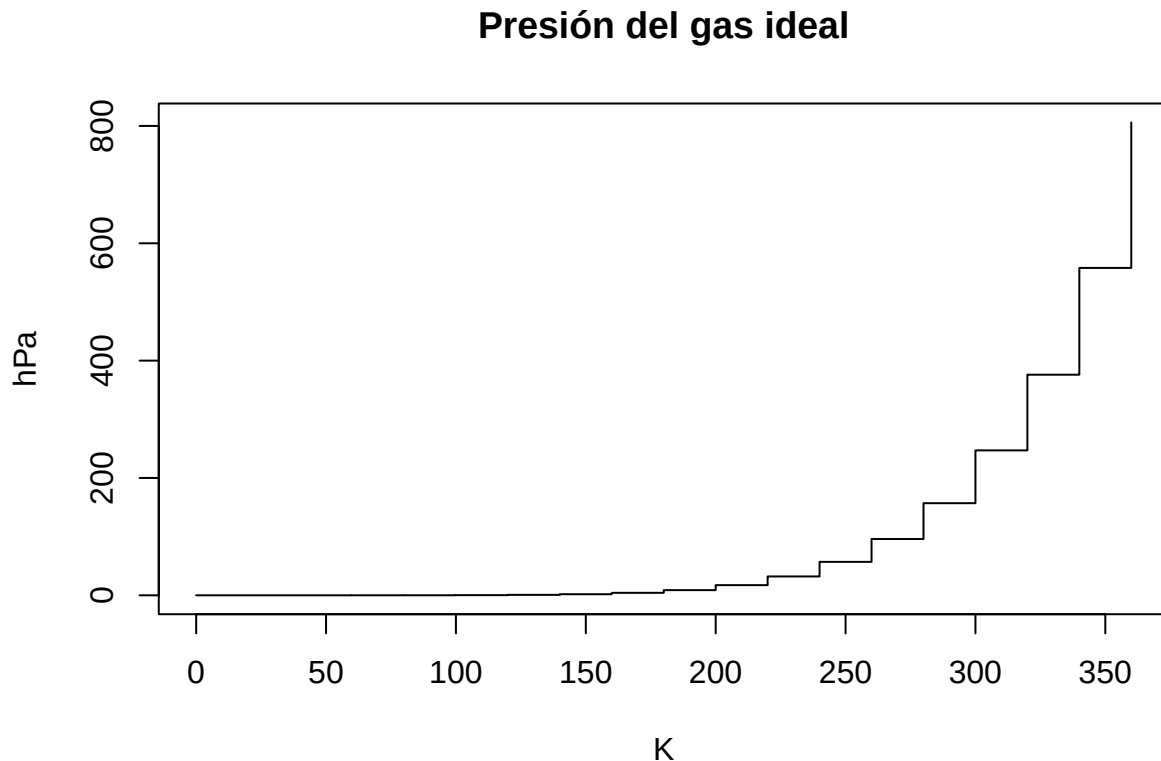
```
##      speed      dist
##  Min.   : 4.0    Min.   :  2.00
##  1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##  Mean   :15.4    Mean   : 42.98
##  3rd Qu.:19.0    3rd Qu.: 56.00
```

```
## Max. :25.0 Max. :120.00
```

Comando plot (Gráficos)

Es un comando que sirve para graficar cualquier fuente de datos que le asignemos: Tenemos la posibilidad de asignar nombres a las variables o hacer cambios del grafico, en caso de no saber el comando podemos escribir **plot** en la consola, a la derecha nos va a abrir una pestaña y ahí tocar Generic X-Y Plotting y ahí nos va a explicar cada comando

```
plot(pressure, type = "s",  
     main = "Presión del gas ideal",  
     ylab = "hPa",  
     xlab = "K")
```



Funciones estadísticas

Otro comando usado es el comando *rnorm*. Este lo que hace es generar datos aleatorios, pero forzando a que esos datos sigan la forma de una Distribución Normal (la famosa campana de Gauss).

Podemos asignarle ciertos parámetros si queremos: *Tamaño de la muestra* **n** (es la cantidad de datos que le pides a R que invente, en este caso 350) *Media* (el valor central, en este caso 22), o *Desviación estándar* (Representa la dispersión o el “error experimental”, en este caso, 5).

```
z1 <- rnorm(350, 22, 5)  
z1
```

```
## [1] 18.806756 13.795480 21.211335 25.698301 24.422535 21.629072 18.779938  
## [8] 21.597634 26.814979 16.414789 16.109991 19.363193 24.564880 17.129495  
## [15] 23.015925 20.483555 16.136092 18.721409 6.672960 21.428802 21.678846  
## [22] 19.051472 22.174730 19.901946 30.828153 16.631194 19.867887 22.027793  
## [29] 24.464438 26.658902 16.144470 16.560908 18.896483 14.283402 28.695718  
## [36] 28.575781 23.851819 27.970775 28.142755 21.084292 24.002535 20.512902
```

```
## [43] 27.399269 20.464391 29.845616 19.801866 17.204028 22.642884 32.375197
## [50] 19.859387 23.578598 27.775907 27.124587 20.112560 15.692252 19.910765
## [57] 22.935194 30.453321 16.473231 25.956413 20.786388 11.704608 27.988081
## [64] 19.945817 28.716522 23.626837 17.870565 26.139501 25.253388 22.634926
## [71] 18.705925 14.112906 23.709364 23.902508 16.492130 21.610891 17.450252
## [78] 23.620542 30.250749 18.741835 15.945583 16.648640 22.508001 21.285883
## [85] 22.553053 21.166330 8.020234 17.440075 14.519117 22.462696 22.865878
## [92] 26.518969 24.590370 24.705814 13.070707 19.796107 23.088614 27.648470
## [99] 29.794884 27.834493 20.636136 22.112498 26.789482 18.485314 27.261578
## [106] 14.220393 24.775531 26.403370 20.936926 22.052341 11.780882 18.868384
## [113] 11.055719 24.480485 17.937995 23.220443 12.227022 16.302046 29.331975
## [120] 27.127734 24.709986 24.540685 15.486801 27.257533 19.226805 29.490803
## [127] 19.131846 19.341838 16.761240 20.933557 18.119769 26.050719 17.263787
## [134] 23.224609 23.609301 23.874968 28.406623 22.715667 20.256989 21.768355
## [141] 30.549203 23.724118 17.608033 24.611584 9.543479 18.405980 19.470313
## [148] 28.480940 17.282821 21.295212 21.000677 31.870412 17.117645 18.858727
## [155] 13.889367 29.322195 19.545247 27.403016 16.744051 29.549781 24.620952
## [162] 18.591731 31.676932 21.495026 17.981075 30.285201 29.764880 27.709827
## [169] 18.382129 28.009582 23.430066 28.677483 26.939113 20.261625 23.236905
## [176] 26.793434 28.181272 15.454330 19.425816 12.917271 12.660885 32.593373
## [183] 24.943192 26.847228 24.518943 30.175696 26.790916 18.188857 14.375896
## [190] 26.777735 4.329805 20.603710 28.048755 21.134495 27.944074 23.281164
## [197] 27.835792 21.442335 23.129367 22.861425 23.465489 18.171128 23.026471
## [204] 28.017530 13.746497 15.137471 12.433310 22.106448 25.450377 26.255414
## [211] 22.896719 19.189862 19.610024 27.480599 17.131409 16.850654 25.966003
## [218] 22.272046 17.667965 15.410528 19.409772 13.733368 20.859810 24.436831
## [225] 18.960569 23.868206 28.623472 23.713435 19.158264 21.714635 22.984685
## [232] 14.221671 17.520932 20.649027 26.411225 32.001468 21.424354 28.057765
## [239] 22.874539 21.355736 15.191438 22.916454 23.139828 11.270597 19.151888
## [246] 22.032613 22.134624 29.507736 12.593490 23.270729 18.115827 28.252194
## [253] 24.262938 13.942889 25.175218 26.933364 23.529659 24.047992 15.639198
## [260] 11.472146 21.017818 24.204187 22.635695 17.123581 25.543168 27.536280
## [267] 17.597169 26.536996 26.035328 22.385956 20.030579 26.649790 20.774000
## [274] 19.095049 23.612070 31.164495 18.279703 21.706240 22.474020 26.672519
## [281] 19.381288 11.843416 24.041539 25.608174 21.675187 27.456110 18.041609
## [288] 24.522173 27.781560 30.179025 20.977047 15.581138 27.871740 27.681267
## [295] 22.130571 22.093072 18.645385 21.606313 27.243474 16.042463 22.796519
## [302] 28.672004 21.844994 25.889250 19.194815 22.487370 22.352472 16.737490
## [309] 22.205358 18.322215 22.112187 24.163520 20.230794 28.252342 24.077865
## [316] 28.178622 27.853280 14.236203 18.274229 21.237390 20.933856 18.169453
## [323] 23.250050 19.178826 28.391791 21.960100 34.783093 22.368839 19.942676
## [330] 25.336407 16.423202 26.345426 21.938860 23.337407 21.632983 27.624415
## [337] 26.584787 20.121288 19.678384 19.464647 30.384826 19.168692 17.456377
## [344] 20.216118 12.890744 24.897055 16.094739 23.457700 19.587553 17.026449
```

```
w1 <- length(z1)
```

```
w1
```

```
## [1] 350
```

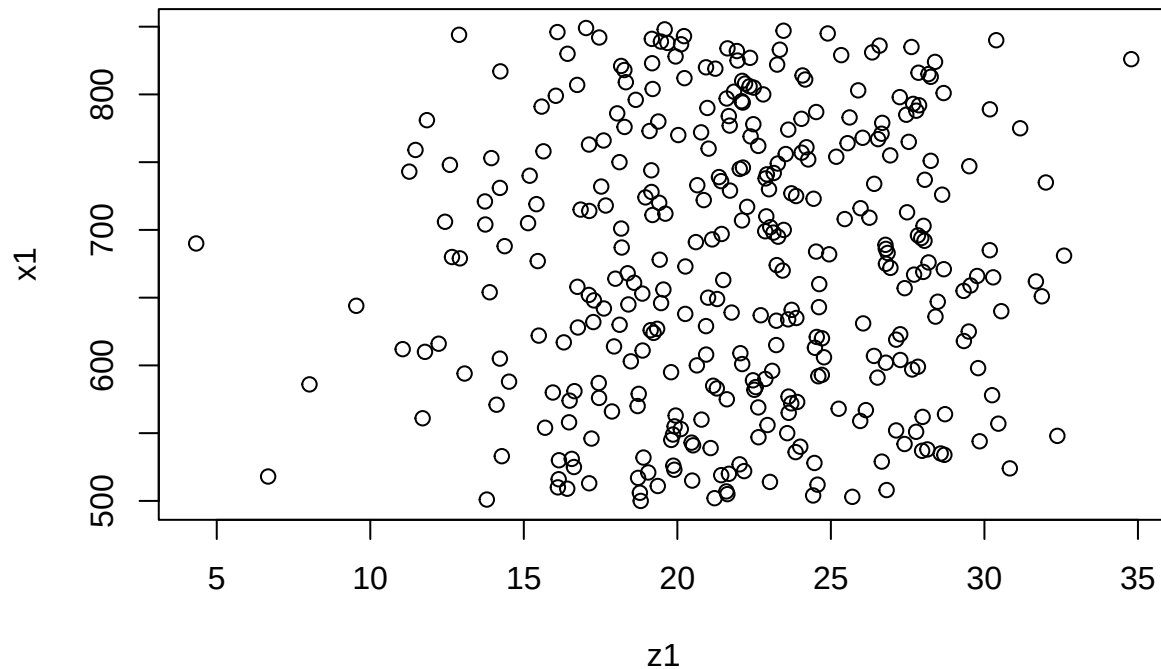
```
x1 <- 500:849
```

```
x1
```

```
## [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
## [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
## [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
```

```
## [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
## [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
## [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

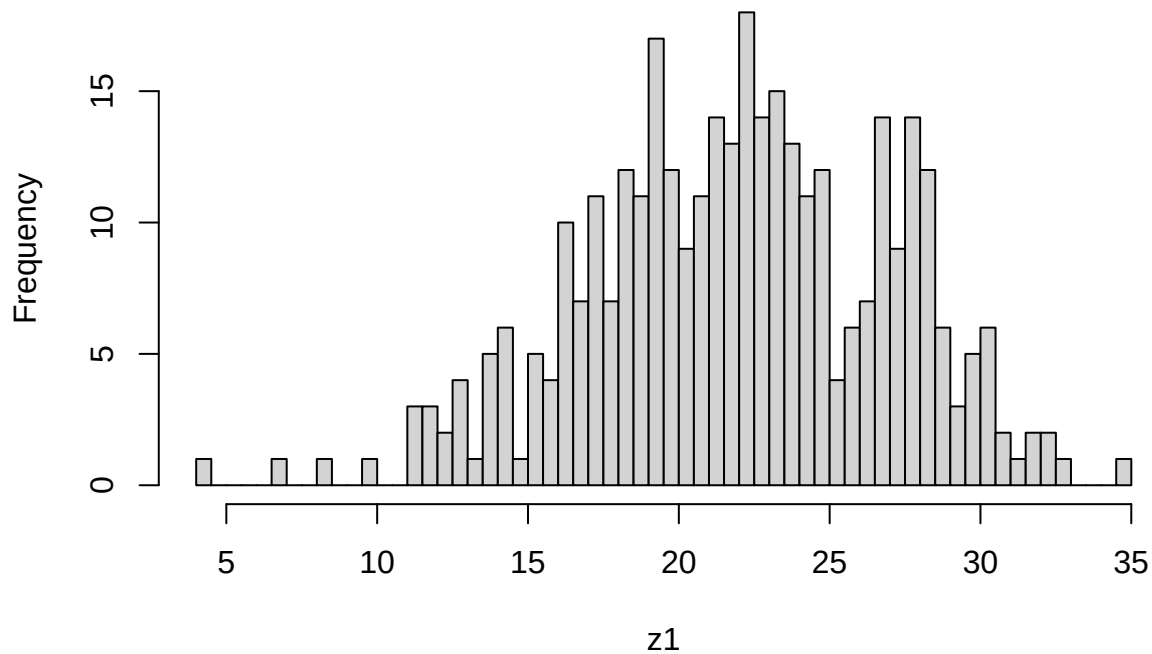
```
plot(z1, x1)
```



El comando `hist()` genera un histograma.

```
hist(z1, main = "Histograma de edades", breaks = 60)
```

Histograma de edades



El comando `density()` calcula la densidad de probabilidad. Es la versión matemática, suavizada y continua del histograma.

```
density(z1,type="b")
```

```
## Warning: In density.default(z1, type = "b") :  
## extra argument 'type' will be disregarded
```

```
##
```

```
## Call:
```

```
## density.default(x = z1, type = "b")
```

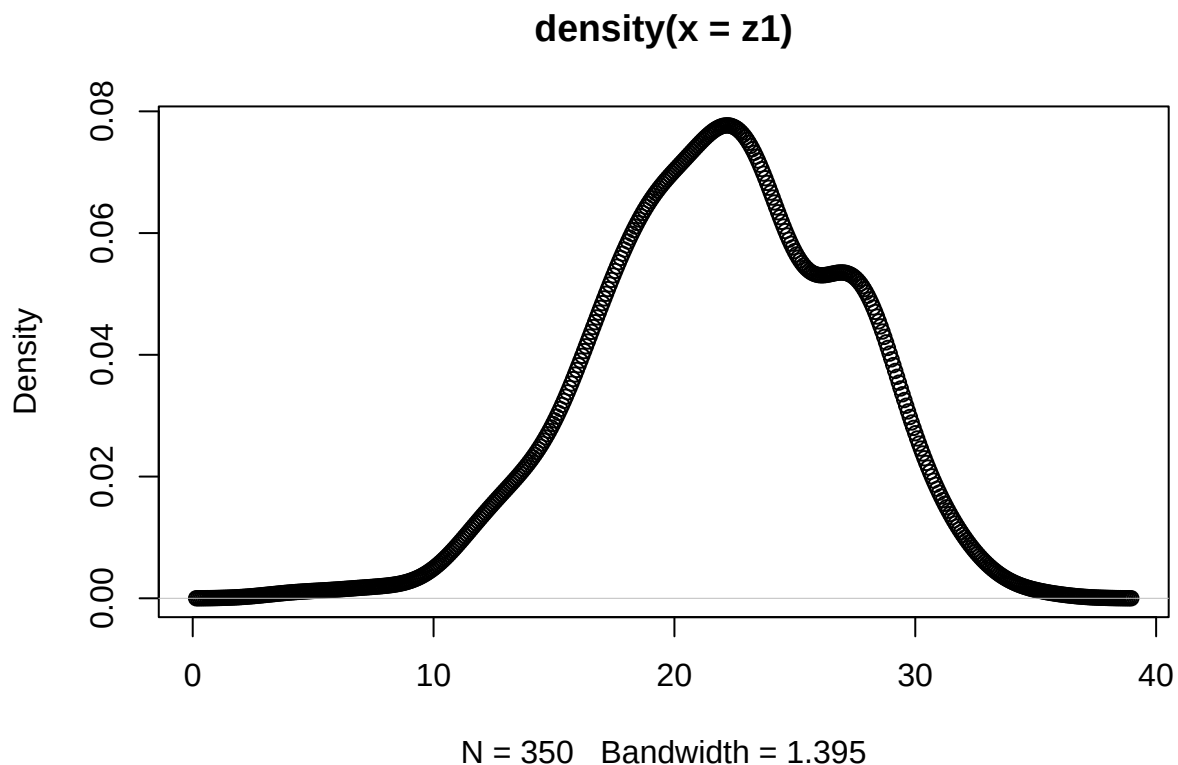
```
##
```

```
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.395
```

```
##
```

```
##      x              y  
## Min.   : 0.144    Min.   :9.124e-06  
## 1st Qu.: 9.850    1st Qu.:1.401e-03  
## Median :19.556    Median :1.359e-02  
## Mean   :19.556    Mean   :2.571e-02  
## 3rd Qu.:29.263    3rd Qu.:5.308e-02  
## Max.   :38.969    Max.   :7.770e-02
```

```
plot(density(z1),type="b")
```



Ejercicio 1: Designación

Consigna: Las ultimas 3 cifras del DNI son 858, crear una variable que tenga ese número

```
DNI <- 858
```

Consigna: Crear un vector del 1 al 858.

```
secuencia_dni <- 1:858
secuencia_dni
```

```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
```

```
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774
## [775] 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792
## [793] 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810
## [811] 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828
## [829] 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846
## [847] 847 848 849 850 851 852 853 854 855 856 857 858
```

Ejercicio 2: Uso de for

Consigna: Calcular la suma de todos los valores del vector *secuencia_dni* usando un for:

```
Total <- 0
valor_final <- length(secuencia_dni)

for (i in 1:valor_final) {
  Total <- Total + i
}

Total

## [1] 368511
```

Ejercicio 3: Python

Consigna: Repetir el ejercicio anterior pero en Python

```
# --- Ejercicio 1 ---
# Consigna: Crear una variable con las últimas 3 cifras del DNI (858)
dni = 858
```

```

# Consigna: Crear un vector (lista) del 1 al 858
# En Python, range(1, 804) genera números del 1 al 858
secuencia_dni = list(range(1, dni + 1))

# --- Ejercicio 2 ---
# Calcular la suma de todos los valores usando un bucle 'for'
total = 0

for i in secuencia_dni:
    total += i

# Mostrar el resultado final
print(f"La suma total es: {total}")

## La suma total es: 368511

```

Ejercicio 4: Medición de tiempo (sys.time)

Consigna: ¿Cuanto tarda en correr el código del ejercicio 2?

Usamos el comando `sys.time`

```

inicio <- Sys.time()
total <- 0
valor_final <- 10000000*length(secuencia_dni)
for (i in 1:valor_final)
total <- total + i
total

## [1] 3.68082e+19

final <- Sys.time()
final-inicio

## Time difference of 2.527267 mins

```

Ejercicio 5: 2da medición de tiempo (tictoc)

Consigna: Aplicar la consigna anterior pero aprendiendo a usar el comando `tictoc`

```

library(tictoc)

# Iniciamos el cronómetro asignándole un nombre descriptivo a la prueba
tic("Tiempo de ejecución")

# Inicio del bloque a medir
Total<-0
valor_final<-length(secuencia_dni)

for(i in 1:valor_final) {
Total<-Total + i
}

# Fin del bloque a medir

```



```

#Se "detiene" el cronómetro.

#Esto imprimirá el tiempo transcurrido en pantalla.
toc()

## Tiempo de ejecución: 0.004 sec elapsed
#Imprimimos la variable Total para verificar que el cálculo matemático se hizo bien
Total

## [1] 368511

```

Ejercicio 6: Secuencia

Consigna: Generar secuencia de 2 en 2 hasta 50000

```

inicio_for <- Sys.time()

A <- numeric(50000)
for (i in 1:50000) {
  A[i] <- i * 2
}

final_for <- Sys.time()
tiempo_for <- final_for - inicio_for

```

Secuencia con función de R:

```

inicio_seq <- Sys.time()

B <- seq(2, 100000, by = 2)

final_seq <- Sys.time()
tiempo_seq <- final_seq - inicio_seq

# Resultados
tiempo_for

```

```
## Time difference of 0.006306887 secs
```

```
tiempo_seq
```

```
## Time difference of 0.001998901 secs
```

Ejercicio 7: Serie de Fibonacci

Consigna: Generar sucesión o serie de Fibonacci hasta superar 1.000.000

```

# Generar Fibonacci hasta superar 1.000.000

fibonacci <- c(0,1)
iteraciones <- 2

while (fibonacci[length(fibonacci)] <= 1000000) {
  nuevo <- fibonacci[length(fibonacci)] + fibonacci[length(fibonacci)-1]
}

```

```

    fibonacci <- c(fibonacci, nuevo)
    iteraciones <- iteraciones + 1
}

fibonacci

## [1]      0      1      1      2      3      5      8     13     21
## [10]     34     55     89    144    233    377    610    987   1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269

iteraciones

## [1] 32

```

Ejercicio 8: Definición matemática recurrente

Consigna: ¿Cuántas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

```

f0<-0 #Valores iniciales
f1<-1
f2<-0
i<-0 #Número de iteraciones inicial
valorfinal<-1000000 #Umbral

while (f2<valorfinal) {

  f2=f0+f1
  f0=f1
  f1=f2
  i<-i+1

}
f2

## [1] 1346269

i

## [1] 30

```

Ejercicio 9: Método Burbuja (Bubble Sort)

Consigna: Compara la performance de ordenación del método burbuja vs el método sort de R.

```

x <- sample(1:20000, 20000)

burbuja <- function(x) {
  n <- length(x)
  for (j in 1:(n-1)) {
    for (i in 1:(n-j)) {
      if (x[i] > x[i+1]) {
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
}

```

```

    }
  }
}
return(x)
}

```

Comparación:

```

system.time({
  res1 <- burbuja(x)
})

```

```

##      user  system elapsed
## 21.916   0.004  22.107

```

```

system.time({
  res2 <- sort(x)
})

```

```

##      user  system elapsed
##   0.001   0.000   0.001

```

Ejercicio 10: Penitencia de Newton

En archivo aparte

Análisis de Rendimiento: Ordenamiento y Agrupación

```

library(microbenchmark)
library(ggplot2)

# Definimos la función del método burbuja (del apunte original)
burbuja <- function(x){
  n <- length(x)
  for(j in 1:(n-1)) {
    for(i in 1:(n-j)){
      if(x[i] > x[i+1]){
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
  return(x)
}

# Generamos una muestra aleatoria moderada para la prueba
set.seed(123)
muestra <- sample(1:1000, 400, replace = TRUE)

# Ejecutamos el benchmarking
mbm_orden <- microbenchmark(

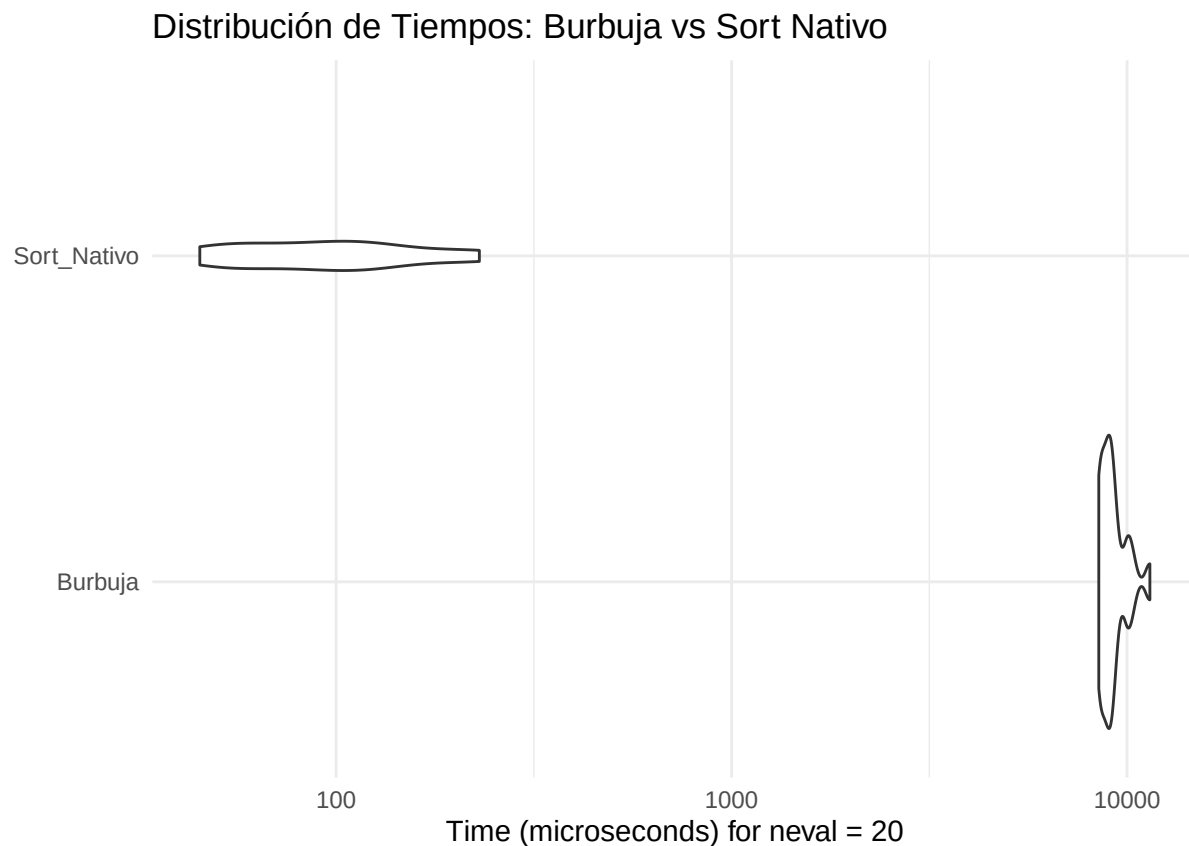
```

```

Burbuja = burbuja(muestra),
Sort_Nativo = sort(muestra),
times = 20 # Repetimos 20 veces para tener una buena distribución
)

# Visualizamos con gráfico de violín
autoplot(mbm_orden) +
  ggtitle("Distribución de Tiempos: Burbuja vs Sort Nativo") +
  theme_minimal()

```



```

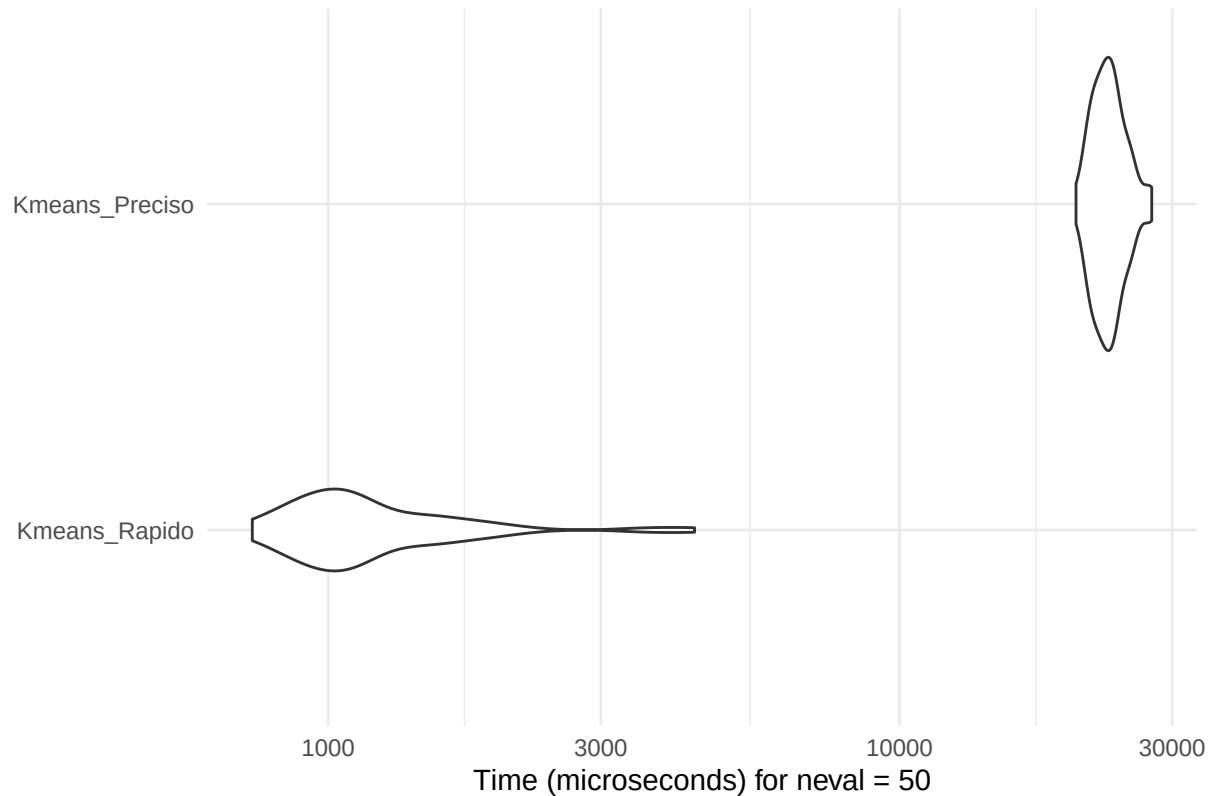
# Generamos una matriz de datos simulada (ej: variables de mantenimiento industrial)
set.seed(42)
datos_industriales <- matrix(rnorm(5000), ncol = 5)

# Benchmarking del algoritmo K-Means
mbm_kmeans <- microbenchmark(
  Kmeans_Rapido = kmeans(datos_industriales, centers = 4, nstart = 1),
  Kmeans_Preciso = kmeans(datos_industriales, centers = 4, nstart = 25),
  times = 50
)

# Visualizamos con gráfico de violín
autoplot(mbm_kmeans) +
  ggtitle("Performance de K-Means: Impacto del parámetro nstart") +
  theme_minimal()

```

Performance de K-Means: Impacto del parámetro nstart



```
# Librerías necesarias
install.packages("ggplot2")

## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.6'
## (as 'lib' is unspecified)
install.packages("cluster")

## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.6'
## (as 'lib' is unspecified)
library(ggplot2)
library(cluster)

# 1. Simulación de datos de sensores (Normalizados)
set.seed(123)
vibracion <- c(rnorm(100, 2, 0.5), rnorm(100, 5, 0.8), rnorm(100, 8, 0.6))
temperatura <- c(rnorm(100, 30, 5), rnorm(100, 50, 7), rnorm(100, 70, 5))
df <- data.frame(vibracion, temperatura)

# Escalamiento (Paso crítico mencionado en tu texto)
df_scaled <- scale(df)

# 2. Ejecución de K-means (k=3)
fit <- kmeans(df_scaled, centers = 3, nstart = 25)
df$cluster <- as.factor(fit$cluster)
```

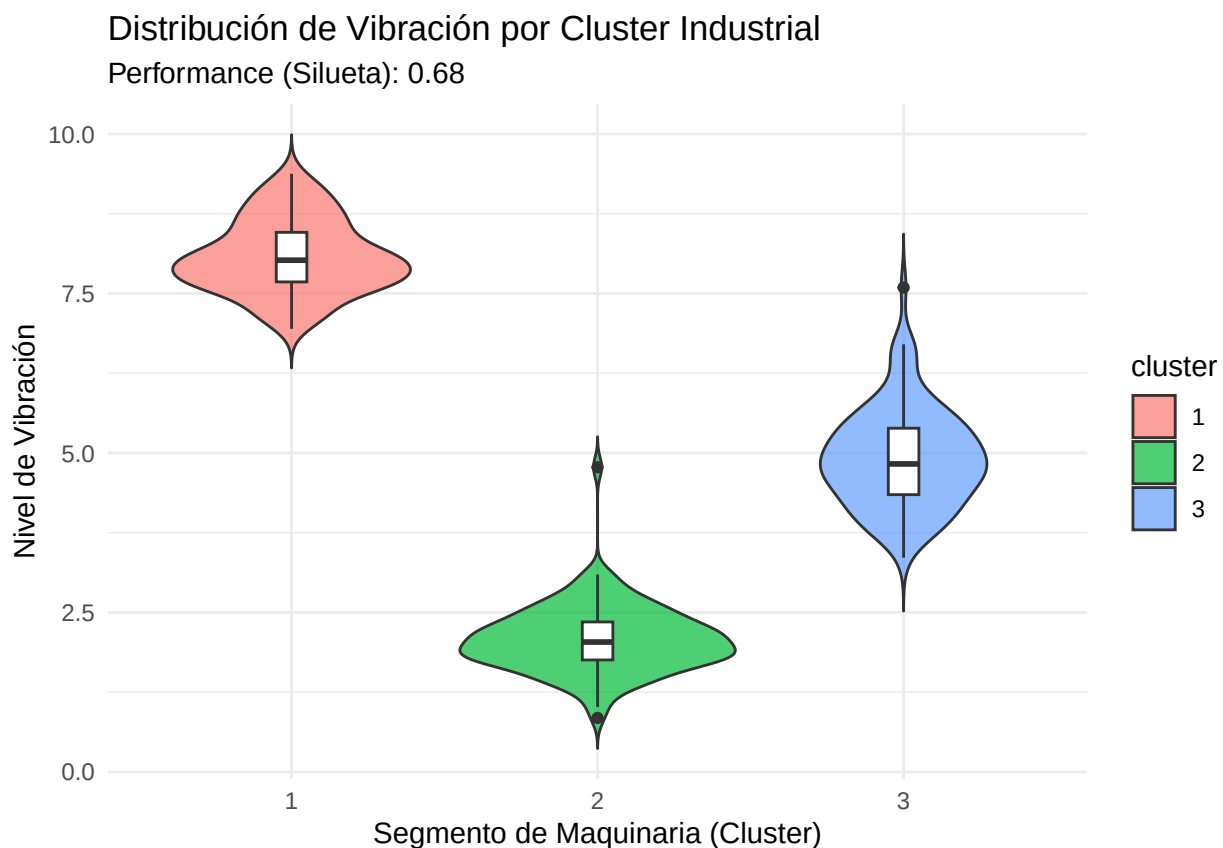
```

# 3. Medición de Performance: Coeficiente de Silueta
sil <- silhouette(fit$cluster, dist(df_scaled))
avg_sil <- mean(sil[, 3])
cat("El coeficiente de silueta promedio es:", avg_sil)

## El coeficiente de silueta promedio es: 0.6808356

# 4. Visualización con Gráfico de Violín
ggplot(df, aes(x = cluster, y = vibracion, fill = cluster)) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  geom_boxplot(width = 0.1, fill = "white") +
  labs(title = "Distribución de Vibración por Cluster Industrial",
       subtitle = paste("Performance (Silueta):", round(avg_sil, 2)),
       x = "Segmento de Maquinaria (Cluster)",
       y = "Nivel de Vibración") +
  theme_minimal()

```



Ejercicio Matrices e Inversa

En este ejercicio, crearemos una matriz cuadrada de 3x3 utilizando el valor de tu DNI (030) y otros valores para asegurar que la matriz sea invertible (que su determinante sea distinto de cero). Luego, utilizaremos la función `solve()` que es la estándar en R para obtener la matriz inversa.

```

dni_val <- 30

datos <- c(dni_val, 10, 15,
          2, 5, 8,
          7, 1, 9)

```

```

A <- matrix(datos, nrow = 3, ncol = 3, byrow = TRUE)

cat("### Matriz Original (A):\n")

## ### Matriz Original (A):
print(A)

##      [,1] [,2] [,3]
## [1,]   30   10   15
## [2,]    2    5    8
## [3,]    7    1    9

determinante <- det(A)
cat("\nDeterminante de la matriz:", determinante, "\n")

##
## Determinante de la matriz: 995

if (determinante != 0) {
  A_inversa <- solve(A)
  cat("\n### Matriz Inversa (A^-1):\n")
  print(A_inversa)

  identidad <- round(A %*% A_inversa, 2)
  cat("\n### Verificación (A * A^-1 = Identidad):\n")
  print(identidad)
} else {
  cat("\nLa matriz es singular y no tiene inversa.")
}

##
## ### Matriz Inversa (A^-1):
##      [,1]      [,2]      [,3]
## [1,] 0.03718593 -0.07537688 0.005025126
## [2,] 0.03819095 0.16582915 -0.211055276
## [3,] -0.03316583 0.04020101 0.130653266
##
## ### Verificación (A * A^-1 = Identidad):
##      [,1] [,2] [,3]
## [1,]    1    0    0
## [2,]    0    1    0
## [3,]    0    0    1

```